N 维 2×…×2 Block 的 Gray-Code Cover-and-Merge SFC（AC 保证）

摘要

本文研究在 d\ge 2 维偶数网格上构造同时满足全覆盖与 Adjacency Continuity（AC，\ell_1=1）的空间填充曲线（SFC）。2×\cdots×2 block 分解：在每个 block 内用 cyclic BRGC 构造 AC Hamiltonian cycle（cover），在 block 相邻图上选取生成树并沿树边以 2-edge splice 合并电路（merge），最终将全局 cycle 剪开得到 SFC path（cut）。本文给出 splice 的局部正确性论证与 portal edge 的存在性下界，并将“matched-portal 是否存在”作为工程闭环的可检验条件：若某次合并找不到匹配门户，则算法返回失败并提供诊断信息。进一步说明该确定性 surrogate）组合，并指出分布对齐与 surrogate gap 校准对稳定训练的必要性。

关键词

空间填充曲线；Gray code；超立方体；Hamiltonian cycle；Adjacency Continuity；cover-and-merge；matched-portal；surrogate modeling

1 引言

空间填充曲线常用于将多维数据映射到一维序列以支持范围查询、缓存友好遍历与压缩等任务。本文关注一种以 AC 为硬约束的构造方案：在保证相邻访问点 \ell_1=1 的前提下，通过可控的局部合并原语（2-edge splice）在块级结构上构建全局曲线。

本文采用分层组织：

- 主文：给出算法定义、关键构造与正确性要点。
- 附录：给出证明细节、断言清单与工程诊断接口。

主要贡献概括如下：

- 在 2×\cdots×2 blocks 上给出统一的 cover-and-merge 构造，并明确每一步 splice 的局部不变量与可验证条件。
- 给出 cyclic BRGC 在任意 face 上 portal edge 存在性的稳健下界，从而保证 splice 的候选集合非空。
- 将高维合并中的 matched-portal 问题以“条件式收敛 + 失败可诊断”的工程闭环方式表述，避免隐含不必要的强假设。
- 给出与学习框架对接的形式化接口：G 生成权重、确定性解码器产出 SFC、E 回归真实指标并通过周期性校准控制 surrogate gap。

1.1 结论（Scope & Claims）

在 d\ge 2 的偶数网格 G=\prod_i\{0,\dots,H_i-1\} 上，取 block_size=2 将网格划分为 2\times\cdots\times 2 超立方体 blocks。

- **Cover**：每个 block 内用 cyclic BRGC 构造一个 AC 的 Hamiltonian cycle。
- **Merge**：在 block 相邻图上选一棵生成树 T，沿 T 的每条边把相邻两个 cycles 用 2-edge splice 合并。
- **Cut**：最终得到一个全局 cycle，剪开任意一条边得到 SFC path。

得到的 SFC 具有：

- **全覆盖**：遍历所有 cell 且不重不漏。
- **全局 AC**：相邻访问点满足 \ell_1=1。
- **可实现（工程闭环）**：每次 splice 只修改常数条边；若找不到可用 splice，则报告“当前实现未找到匹配门户”，并输

1.2 问题定义（Problem）

SFC 是对所有 cell 的排列 \pi=(\pi_0,\dots,\pi_{N-1})。AC 要求

$$\|\pi_{t+1} - \pi_t\|_1 = 1.$$

典型目标（如 range query）是最小化查询映射到 1D 后的区间段数等指标。

1.3 设计选择：block_size=2 与 cyclic BRGC

2\times\cdots\times 2 block 与超立方体图 Q_d 同构；cyclic BRGC 给出标准、可复现的 Hamiltonian cycle，且每步 1-bit 翻转 \Rightarrow \ell_1=1。

更重要的是：BRGC 的环在每个 face 上都存在 face 内边（portal edges），从而允许在相邻 blocks 间做“切边 + 接边”的 splice（细节与计数推导见附录）。

1.4 跨 block 的几何配对（Geometric pairing）

相邻 blocks A 与 B=A+\mathrm{sign}\cdot e_k，定义 face 上的几何配对

$$\phi(x)_k = x_k + \text{sign}, \qquad \phi(x)_j = x_j\,(j \neq k),$$

则必有

$$\|x - \phi(x)\|_1 = 1.$$

这保证了我们“跨 face 接边”时仍满足 AC。

1.5 合并原语：2-edge splice

设当前有两个相邻 components（各自是一条 cycle）C_1，C_2。

1. 在 C_1 面向 C_2 的 face 上找一条 face 内边 (u,v)；
2. 在 C_2 对应 face 上找 (u',v')，并满足 u'=\phi(u),\ v'=\phi(v)；
3. 删除 (u,v)、(u',v')，添加跨边 (u,u')、(v,v')。

由于 splice 仅做“删两条 + 加两条”且新增跨边由 \phi 保证为 \ell_1=1，因此 splice 同时保持覆盖与 AC，并把两个 cycles 合并为一个（component 数减 1）；沿生成树重复即可得到全局 cycle。

1.6 解码流程（Decoding）

1. 给 block 相邻边赋权 w，取最大生成树 T；
2. 每个 block 内生成 cyclic BRGC cycle；
3. 沿 T 逐边做 2-edge splice 合并；
4. 得到全局 cycle 后剪开成 path。

1.7 学习结合（NeuralSFC-style）

Generator 输出 w（或 residual：w=w_0+\Delta w），Decoder 用上面确定性流程把 w 解码成 SFC；Evaluator 用真实指标（如 intervals）回归 surrogate 分数，交替训练并用真实解码样本校准。

1.7.1 设定 B：场景自适应曲线（推理阶段）

面向 range query 的 workload-aware 场景下，更自然的设定是：给定场景统计 scene，由 G(scene) 输出一条**定制曲线**。推理阶段可写为

scene \xrightarrow{G} W \xrightarrow{\mathrm{Decode}} \pi(scene)。

这里 \mathrm{Decode} 是本文给出的确定性 cover-and-merge 解码器（MaxST + cyclic BRGC + 2-edge splice + cut），因此即使 G 是场景自适应的，最终输出也始终满足：

- 全覆盖（遍历所有 cell 且不重不漏）；
- 全局 AC（相邻访问点 \ell_1=1）。

1.7.2 为什么仍沿用“连续参数 + 解码器”的套路

直接让 G 输出离散排列 \pi 往往训练不稳且容易生成非法结构。更推荐让 G 输出连续参数（例如 block 相邻边权重 W，以及 block 内 orientation/入口出口选择的连续松弛），再经 \mathrm{Decode}(W) 还原为合法曲线。这样学习只负责“偏好/排序”，而合法性（AC + 覆盖）由解码器保证。

1.7.3 G(scene) 的输入：避免全量点集，使用可计算的场景表征

场景 scene 不建议直接输入全量点集（可变长且代价高），更可落地的输入是固定形状的统计特征，例如：

- **密度网格**：\rho(c)（cell 内点数或概率质量）；
- **查询热度网格**：h(c)（cell 被窗口覆盖/命中的概率）；
- **窗口大小分布统计**：每维窗口边长的均值/方差或离散 bins。

你提出的“cell-cell 联合概率密度图”在信息上更贴近 #intervals，但完整 joint P(c_i,c_j) 是 O(N^2)，在 d\ge 3 时不可用。MVP 方案可使用**局部稀疏 joint**：

- **四邻域/2d 邻域联合概率**：仅存储 J(c,c+\delta)，其中 \delta 取 2d 个单位邻域方向；该特征规模 O(N)。

该局部 joint 主要捕捉“相邻 cell 在同一查询中共同出现”的倾向，能为 G 提供可计算的局部共现信号。若希望兼顾更大窗口，可在 \delta 中加入少量多尺度位移（例如轴向 \pm2,\pm4,\pm8），其复杂度仍为 O(N)。

1.7.4 是否需要 block 级全局建模，以及多尺度分块的 AC 风险

关于“全局聚簇性（降低 #intervals）是否必须上 block 级”：这不是硬性要求，但通常是控制大窗口性能的主要杠杆。直观 #intervals 往往由“区域/块在 1D 上的排列布局”决定，而不仅是局部一步如何走。

你提出“按查询负载对空间进行多尺度划分”的方向是合理的，但需要注意：若允许**不同尺度 block 之间直接拼接**，接口点很可能无法保证是网格相邻，从而破坏 AC。为避免该风险，工程上更稳的做法是：

- **AC 的约束始终在最细 cell 网格上定义**：即最终曲线永远由相邻 cell 步组成；更大尺度 block 只用于组织/建模与参数化，而不改变“合法连接必须发生在共享 face 的相邻 cell 对上”的规则。
- **优先固定最小 block（block_size=2）作为唯一可 splice 的原子单元**：多尺度只作为“上层聚类/先验”，再投影回 2×\cdots×2 原子块的权重与 orientation 决策。

在该约束下，多尺度带来的“得不偿失”风险会显著降低：你可以保留场景自适应与全局布局信号，同时不牺牲 AC 的可验证性。

1.8 多维推广中的卡点：total correctness（matched-portal 必存在/必终止）

需要强调：我们在多维度空间（一般 d\ge 3）遇到的“卡点”并**不是** AC 或覆盖失败，而是 total correctness（必然终止/必然成功合并）尚未获得无条件证明。

已可严格保证（partial correctness）：

- **覆盖（coverage）**：若合并流程成功结束，则最终 SFC 覆盖所有 cells 且不重不漏。
- **AC**：cyclic BRGC 的相邻码对应 \ell_1=1 相邻 cell；跨 block 的几何配对 \phi 亦保证新增跨边 \ell_1=1。
- **splice 的局部正确性**：每次 2-edge splice 执行“删两边加两边”，保持 2-regular 不变量（从而保持 Hamiltonian 结构的可行性），并使 component 数减少 1。

唯一尚未无条件证明的是（total correctness）：

- **对生成树的每一步合并，在其发生时刻 matched-portal 必存在**。

更具体地说，A7.2 已证明每个共享 face 上至少存在 portal edge（候选边非空），但在 d\ge 3 时，一条超立方体边可能同时属 faces，导致一次 splice 对某个 face 上 portal 的选择会与其他 faces 的可用 portal 发生耦合；因此“存在 portal edge”并不自动推出“存在一对几何配对后一一对应的 matched portals”，从而无法直接得到“沿任意生成树顺序

本文因此将收敛性表述为条件式（见 A10.3）：若每次合并时刻都能找到 matched portals，则 component 数沿生成树边数严格下降并最终收敛为 1；否则算法应报告失败并输出诊断信息（见 A7.4 与 A13.2）。

1.9 2D 复现：本仓库最小实现（建议先跑通）

本节把上面的“Cover + Tree + Merge + Cut”在 **2D** 情况下落地到本仓库已有实现：

- Cover：basic/context.py::build_circuits_graph（每个 2×2 block 初始 4-cycle）
- Tree：basic/context.py::build_dual_graph + networkx.minimum_spanning_tree（对偶图生成树）
- Merge：basic/context.py::apply_mst（对每条树边做“删两条 + 加两条”的 2-edge splice）
- Cut/Extract：删除一条边后用 nx.all_simple_paths 抽取一条 Hamiltonian path

1.9.1 一键生成与验收（AC + 全覆盖）

仓库根目录提供了最小脚本 reproduce_final_2d.py，用于：

- 随机生成一张 H×W 的灰度矩阵（要求 H,W 为偶数）
- 调用上述确定性解码流程生成一条 SFC
- 断言验证：
  - len(path)=H・W
  - len(set(path))=H・W
  - ∀t, ||path[t+1]-path[t]||_1 = 1

运行方式：

```
export PYTHONPATH=".:$PYTHONPATH"
python reproduce_final_2d.py --h 32 --w 32
```

若需要输出可视化与 .npy：

```
export PYTHONPATH=".:$PYTHONPATH"
python reproduce_final_2d.py --h 32 --w 32 --save --out ./output/reproduce_final_2d
```

输出文件说明：

- sfc.png：按图结构绘制（networkx 风格，包含节点与边），便于检查连接关系。
- ordered_curve.png：按访问顺序绘制的折线（可视上更接近论文/comparison.gif的“单条曲线”）。

1.9.2 与附录的对应关系

该 2D 落地版本等价于把附录中的一般维度 splice 过程特化到 2D：

- 2D 时每次合并只发生在共享边界（face 即边），实现上由 apply_mst 在对应 2×2 block 边界执行“删旧边 + 加跨边”。
- 由于 2D 情况下“边同时属于多个 face”的耦合不存在，matched-portal 的高维卡点不会在该最小复现里出现；因此建 **AC/全覆盖** 的工程闭环跑通，再讨论 N 维的 portal 管理与收敛性假设。

---

2 附录（实现与证明细节）

2.0.1 A1. 范围与结论（Scope & Claims）

在如下前提下：

1. **维度**：d ⩾ 2。
2. **网格**：G = \prod_i \{0,\dots,H_i-1\}，且每个 H_i 为偶数。
3. **分块**：block_size = 2，每个 block 是 2×...×2 的 d 维超立方体。
4. **局部覆盖（Cover）**：每个 block 内使用 **cyclic BRGC（binary reflected Gray code）** 构造 **Hamiltonian cycle**（闭环，且每条边为 1-bit 翻转，因此 \ell_1=1）。
5. **全局拓扑（Tree）**：在 block-level 相邻图上，依据边权 w 取一棵生成树 T（通常用 **Maximum Spanning Tree**；实现上可用 \mathrm{MST}(-w)）。
6. **合并（Merge）**：沿 T 的每条树边以 **2-edge splice** 合并电路；每次 splice 前执行 A7.4 的 **matched-portal 搜索**，若找到匹配门户则 splice 可执行，并通过断言验证其合法性。

则可构造 SFC \pi 满足：

- **全覆盖**：遍历所有 cell 且不重不漏。
- **全局 AC**：\forall t,\ \|\pi_{t+1}-\pi_t\|_1 = 1。
- **可合并性（工程闭环条件式）**：对生成树 T 的每条树边，若在当次合并时存在一对 matched portals，则可执行一次合法 splice 且 component 数减少 1。沿 T 重复执行 m-1 次 splice（m 为 block 数），最终 component 数收敛为 1（得到单一全局 cycle）；剪开任意一条边得到 path 形式的 SFC。

  注：matched-portal 搜索是“找到则执行、找不到则报告”的工程步骤；本文对“必收敛”结论采用**条件式**表述，以

2.0.2 A2. 问题定义（Problem Definition）

1. **网格**：G = \prod_{i=1}^d \{0,1,\dots,H_i-1\}，且 d\ge 2。
2. **SFC**：\pi = (\pi_0,\dots,\pi_{N-1})，其中 N=\prod_i H_i。
3. **AC（Adjacency Continuity）**：相邻 cell 的 Manhattan（\ell_1）距离为 1：

$$\|\pi_{t+1} - \pi_t\|_1 = 1.$$

4. **目标（range query 示例）**：最小化

$$C(\pi; Q) = \mathbb{E}_{R\sim Q}[\mathrm{Intervals}(I(R))], \quad I(R) = \{\mathrm{pos}_\pi(x) \mid x \in R\}.$$

2.0.3 A3. 设计选择与原因（Why BRGC + smallest blocks）

2.0.3.1 A3.1 为什么选择 BRGC / Gray code

- cyclic BRGC 在 {0,1\}^d 上给出一个**规范、可复现**的 Hamiltonian path/cycle 构造。
- 相邻码字翻 1 bit \Rightarrow 网格上 \ell_1=1。
- 更关键：该 cycle 在每个 face 上提供 **face 内边（portal edges）**，使 2-edge splice 的“切边+接边”策略可持续执行（A7.2）。

2.0.3.2 A3.2 BRGC 的“标准性”价值（工程/科研收益） 尽管超立方体图 Q_d 上 Hamiltonian cycle 并不唯一，选择 BRGC 的价值在于：

- **可重复性**：不同实现得到一致结果。
- **可调试性**：中间状态可精确对照理论结构。
- **可比较性**：算法对比不受“任意选环”自由度影响。
- **可验证性**：单元测试容易构造（尤其 face portal 的检测、deg=2 断言、\ell_1=1 检查等）。

2.0.3.3 A3.3 为什么选最小 block（block_size=2）

- block 结构严格等价于超立方体图 Q_d，face 等价于 Q_{d-1}。
- 跨 block 的几何配对 \phi 简化为“某一维坐标 \pm 1”。
- portal existence 与 splice 正确性证明维度无关、形式最简。
- splice 可局部 O(1) 更新，并能用“deg=2”“新增边 \ell_1=1”快速断言与调试。

2.0.4 A4. 数学对象与符号（Notation）

- block_size=2，B_i=H_i/2。
- block 坐标：b \in \prod_i \{0,\dots,B_i-1\}。
- block cells：Cells(b) = \{2b+s \mid s\in\{0,1\}^d\}。
- block 相邻图：G_{\mathrm{blk}}=(V_{\mathrm{blk}},E_{\mathrm{blk}})。
- 每条相邻边带属性 direction=(k,\mathrm{sign})：
  - k \in \{0,\dots,d-1\} 为相邻维度；
  - \mathrm{sign}\in\{+1,-1\} 表示 B = A + \mathrm{sign}\cdot e_k。

2.0.5 A5. 端到端解码流程（Decoding Overview）

1. **Weight assignment**：得到 block 边权 w_e。
2. **MaxST**：取最大权生成树 T（实现上可用 \mathrm{MST}(-w)）。
3. **Cover**：每个 block 内构造 BRGC Hamiltonian cycle。
4. **Merge**：沿 T 对相邻 components 执行 2-edge splice（先 matched-portal 搜索）。
5. **Cut**：将最终单一 cycle 剪开得到 SFC path。
6. **Evaluate**：计算 \#\mathrm{intervals} 等指标。

2.0.5.1 A5.1 splice 的执行顺序（不影响正确性）

只要每次 splice 合法，T 的边以任何顺序执行都会把 component 数从 m 降到 1。工程上推荐：

- **BFS 顺序**：便于可视化与调试中间状态。
- **DFS 顺序**：递归实现方便。
- **任意顺序**：可行，但更难定位中间状态 bug。

2.0.6 A6. Boundary pairing：跨 block 的几何配对（Geometric Bijection）

相邻 blocks A 与 B 满足 B = A + \mathrm{sign}\cdot e_k。令共享维度为 k，符号为 \mathrm{sign}。

在 cell 层面的几何配对定义为 \phi：

- \phi(x)_k = x_k + \mathrm{sign}
- \phi(x)_j = x_j （j\neq k）

因此对任意 x（位于 A 朝向 B 的那个 face 上）都有：

$$\|x - \phi(x)\|_1 = 1.$$

强调：该结论与 Gray code 的“排序”无关，是纯几何事实；排序仅用于实现的规范化、可复现与调试便利。

2.0.7 A7. Merge 的正确操作：2-edge splice（两门户）

2.0.7.1 A7.1 cycle 的工程表示（推荐）

建议用 2-regular 图表示 cycle：每个顶点 v 存两个邻居 nbrs[v]=\{a,b\}。splice 仅改动 4 个点邻接即可 O(1)。

2.0.7.2 A7.2 Portal edge 的存在性与数量级（稳健版本）

定义（Portal edge）：对某个 face F，若当前 cycle 上存在边 (u,v) 满足 u,v\in F，则称 (u,v) 为 portal edge。

引理（Portal existence，d\ge 2）：在 {0,1\}^d 上由 cyclic BRGC 构造的 Hamiltonian cycle 中，对任意维度 k 与侧 s\in\{0,1\} 的 face x_k=s，至少存在一条 portal edge。

数量级说明（形式化且稳健）：令 S 为某个 face 的顶点集合，|S|=2^{d-1}。设 cycle 在 cut (S, V\setminus S) 上的穿越次数为 c（等价于该维度对应 bit 在整条 cycle 上的翻转次数）。对任意 Hamiltonian cycle 恒有：

$$E_{\text{in}}(S) = |S| - \frac{c}{2}.$$

对 cyclic BRGC，任意维度对应的翻转次数满足 c \le 2^{d-1}，因此：

$$E_{\text{in}}(S) \geq 2^{d-1} - 2^{d-2} = 2^{d-2} \geq 1 \quad (d \geq 2).$$

从而每个 face 上至少存在 portal edge，且 portal edges 在所有 face 上都有 \Omega(2^{d-2}) 的下界。

补充说明（为何对 cyclic BRGC 有 c \le 2^{d-1}）：对标准 cyclic BRGC，可用生成式

$$g(t) = t \oplus (t \gg 1), \qquad t = 0, \ldots, 2^d - 1,$$

并将 (g(2^d-1),g(0)) 作为闭环最后一条边。

对非闭环边 (g(t),g(t+1))，其翻转的 bit 位置等于 \mathrm{lsb}(t+1)（最低位 1 的位置）。因此对任意 k\in\{0,\dots,d-2\}，第 k 位在整条闭环上的翻转次数为

$$c_k = \#\{n \in [1, 2^d - 1] : \text{lsb}(n) = k\} = 2^{d-1-k} \leq 2^{d-1}.$$

对最高位 k=d-1，序列内部翻转 1 次（当 n=2^d-1），闭环边再额外翻转 1 次，故 c_{d-1}=2。因而对 cyclic BRGC 任意维度都有 c\le 2^{d-1}，最坏情况发生在 k=0 时 c=2^{d-1}，从而

$$E_{\text{in}}(S) = |S| - \frac{c}{2} \geq 2^{d-1} - 2^{d-2} = 2^{d-2} \ (\geq 1,\ d \geq 2).$$

2.0.7.3 A7.3 2-edge splice（操作定义）

要合并两个相邻 components C_1,C_2（沿维度 k 相邻，符号 \mathrm{sign}）：

1. 在 C_1 的共享 face 上选 portal edge (u,v)；
2. 在 C_2 的对应 face 上选 portal edge (u',v')，满足：
   - u' = \phi(u)，v' = \phi(v)；
3. 执行 splice：
   - 删除 (u,v)、(u',v')；
   - 添加 (u,u')、(v,v')。

断言（每次 splice 后必检）：

- **A0（边存在性）**：删除前必须确认 (u,v) 与 (u',v') 当前确为 cycle 边：u\in nbrs[v] 且 u'\in nbrs[v']。
- **A1（2-regular）**：\forall x,\ \deg(x)=2。
- **A2（AC）**：新增跨边满足 \ell_1=1（由 \phi 的定义可直接验证）。
- **A3（component 数减少）**：component 数减少 1（DSU 记录）。
- **A4（无自环/重边）**：新增边满足 u\neq u'、v\neq v'，且新增后 u'\notin nbrs[u]、v'\notin nbrs[v]（避免产生重复边或与现有边冲突），且新增边不等于被删除的边（避免出现“删了又加回去”的退化操作）。

  实际实现中强烈建议先按 A7.4 做 matched-portal 搜索得到可用的 (u,v) 与 (u',v')，再执行 splice。

2.0.7.4 A7.4 Matched-portal 搜索（工程闭环，强烈推荐）

对树边 e=(A,B,k,\mathrm{sign})，在当前 component C_1 的共享 face 上枚举候选 portal edges (u,v)（即 u,v\in F_A 且 (u,v) 确为当前 cycle 边）。对每个候选，令：

$$(u', v') = (\phi(u), \phi(v)).$$

检查 (u',v') 是否为 component C_2 当前 cycle 的真实边（例如用 2-regular 表示时，检查 u' \in

nbrs[v']）。若是则执行 splice；否则继续枚举下一个候选 portal。

若枚举完仍失败，则报告“matched-portal 不存在/实现错误”并输出用于调试的信息（推荐至少输出：k,\mathrm{sign}、顶点集合、C_1 face 内候选 portal 数量、以及 C_2 face 内实际边数量）。实践中这通常意味着：

- direction/sign 识别错误或低/高侧 face 判定错误；
- \phi 写反（\pm 1 方向错误）；
- 先前 splice 破坏了 2-regular 不变量（deg\neq 2）；
- component 归属（DSU）错误导致在错误的 cycle 上找 portal。

注：定理 A10.3 的收敛性是在 “对每条树边合并时刻存在 matched portals” 的假设下成立； 而 A7.2 已证明对任意共享 face 至少存在 portal edge（且数量有稳健下界）。 因此，若 matched-portal 搜索失败，说明“当前实现未找到一对匹配门户”。其原因可能包括：face/方向判定错误、\phi 实现错误、2-regular 不变量被破坏、或该合并时刻确实不存在 matched portals；建议按上述诊断项逐一排查。

2.0.8 A8. Cut：从全局 cycle 得到 SFC path

最终得到一个全局单一 cycle。剪开任意一条边即可得到一条 SFC path，并保持 AC。

策略说明（不影响正确性，但影响性能/可重复性）：

- **随机剪开**：最简单。
- **剪开“权重最小/最弱”的边**（启发式）：可能略提升 query 指标。
- **固定位置剪开**（如 corner cell 附近）：便于复现实验与对比。

2.0.9 A9. 与学习结合（Generator/Evaluator）

2.0.9.1 A9.1 Generator 与 residual 模式（baseline） Generator 输出 w 或 \Delta w（residual）：

- w = w_0 + \Delta w

其中 w_0 可取：

- **均匀 baseline**：w_0=1（最朴素可复现）。
- **启发式 baseline**：基于 block 热度/密度的简单公式。
- **预训练 baseline**：历史 checkpoint 的权重均值或固定模型输出。

Residual 的优势：

- **更快收敛**：从合理先验出发。
- **更稳定**：\Delta w 的尺度更小、分布更集中。
- **更可解释**：\Delta w 表示相对 baseline 的调整量。

2.0.9.2 A9.2 Evaluator（surrogate）与校准 Decode(w) 不可微：MaxST → splice → SFC → \#\mathrm{intervals}。用 Evaluator E(S,w) 回归真实得分 y_{\mathrm{true}}=-C(\mathrm{Decode}(w);Q) 作为 surrogate；关于输入分布对齐与周期性真实校准见 A9.4–A9.5。

2.0.9.3 A9.3 2D 解析式 baseline 权重（Dafner-style）的语义 在本仓库的 2D 实现中，w 是 block-level 对偶图 G_{\mathrm{blk}} 上的边权，用于 MaxST/MST 选择 block 的合并关系；它不直接对应像素权重。

以两个相邻 2×2 blocks 的一次合并候选为例，仓库中存在一个常用的解析式 baseline（实现上见 basic/context.py::calc_weight(..., advanced=False) 与 utils/misc.py::WeightsAssigner.calc_weight）：

$$w_e = u + v - e - f.$$

其中 u,v 代表 splice 后倾向保留/新增的跨界连接项（希望小，鼓励跨界连接发生在平滑区域），e,f 代表 splice 时倾向断开的 block 内部连接项（希望大，鼓励“切断”发生在强边界处）。因此该 baseline 体现了“让曲线在平滑区域跨块、避开强边界”的启发式。

2.0.9.4 A9.4 Evaluator 的输入分布对齐（distribution matching）

Evaluator E 的目标是逼近不可微目标函数

$$y_{\text{true}}(S, w) = \text{Metric}(S, \text{Decode}(w)).$$

若训练时 E 仅在某一类 w（例如仅在 baseline 附近）拟合，而推理/优化时 Generator G 会产生不同分布的 w_{gen}，则 E 的误差可能在关键区域显著放大，从而诱发 G 被错误梯度驱动。

工程上应显式做分布覆盖，常见策略包括：

- **Baseline-centered augmentation**：围绕 w_0（例如 w_0=weights_{dafner}）构造扰动样本 w_{tgt}（加噪、缩放、符号翻转、混合等）。
- **On-policy alignment**：训练 E 时额外喂入当前 G 产生的 w_{gen}（通常对 G detach），形成在线对齐的数据闭环。
- **Replay / curriculum / active sampling**：缓存历史 w_{gen}、逐步扩大扰动强度、以及有意采样靠近 MaxST 结构发生变化的边界区域（tie/换边附近）。

2.0.9.5 A9.5 surrogate gap 与周期性真实校准（data aggregation / DAgger-style）

训练时 G 实际优化的是 E(S,w)，而研究目标是 y_{\mathrm{true}}(S,w)；两者的系统性偏差可称为 surrogate gap。若缺乏校准，G 可能利用 E 的误差而在真实指标上退化。

缓解方式是引入周期性真实校准：定期对当前 w_{gen} 执行一次真实解码 \mathrm{Decode}(w_{gen}) 并计算 y_{\mathrm{true}}，将 (S,w_{gen},y_{\mathrm{true}}) 作为新监督数据回灌到 E 的训练集中（数据聚合 / DAgger 风格），以缩小 surrogate gap 并稳定训练。

2.0.10 A10. 全局正确性证明（Sketch，论文式：定理 + 假设）

2.0.10.1 定理 A10.1（Cover 正确性）

对任意 block，cyclic BRGC Hamiltonian cycle 覆盖 block 内全部 2^d 顶点，且每条边满足 \ell_1=1。

2.0.10.2 定理 A10.2（Splice 保持覆盖与 AC）

对两个顶点集合不交且分别为 AC Hamiltonian cycle 的 components，若存在 matched portals 并执行一次 2-edge splice，则：- 顶点覆盖保持（不删点不复制点）；- 新增跨边均为 \ell_1=1（由 \phi 的几何性质）；- 结果仍为 2-regular（deg=2）结构，从而仍是若干 cycles 的并。

2.0.10.3 补充说明（从 deg=2 约束推导 2-edge splice 与“共享 face”约束）

从“本源视角”看，Hamiltonian cycle（或最终 SFC path）要求每个顶点满足入度/出度各为 1（无向图中等价为 deg(v)=2，path 的两个端点为 deg=1）。因此当我们希望在 component C_1 中的某个顶点 u 上新增一条跨 block 的连接边时，若不删除 u 原 cycle 中的一条 incident 边，则 deg(u) 将从 2 变为 3，破坏 Hamiltonian 结构。故在 C_1 侧必须“断开一条旧边”以腾出端 cycle 合并为一个 cycle 并保持所有顶点度数合法，最小的局部正确操作正是“删两条 cycle 边 + 加两条跨边”的 2-edge splice（分别在 C_1 与 C_2 上各删一条边、再加两条跨边），从而保持 deg=2 不变量并使 component 数减少 1。

进一步地，为保持全局 AC（\ell_1=1），跨 block 新增边必须连接两侧网格中 Manhattan 距离为 1 的点；而相邻 blocks 的所有 \ell_1=1 跨界点对，恰好由两者的共享 face 上的点一一对应（由几何配对 \phi 给出）。因此跨边的端点必然落在共享 face 上，2-edge splice 中被删除的“端口边”也必然是共享 face 内的一条 cycle 边。这说明“以顶点端口为中心”的描述与“以共享 face 上的 portal edge（face 内 cycle 边）为中心”的描述是等价的：portal edge 只是将“两个端点同时开口并形成可合并的 cycle 结构”压缩为一个可检索的边对象。

注意：在 d\ge 3 时，一条超立方体边同时属于多个 face（几何上 face 的交集包含 edge），因此一次 splice 选择删除的 portal edge 虽然位于当前合并使用的 face 上，但可能同时也是其他 face 的 portal 候选，从而影响后续合并的可用性。这也是A10.3需要matched-portal假设（或额外的选边策略/回溯机制）才能给出“必

2.0.10.4 定理 A10.3（Tree-merge 收敛：在 matched-portal 假设下） 设 block 数为 m，T 为 block-level 生成树（|T|=m-1）。**假设**：对 T 的每条树边 e，在其合并时刻，边两端当前 cycles 上存在一对 matched portals。则对每条树边执行一次合法 2-edge splice 会使 component 数减少 1，并保持 2-regular 与 AC 不变量。执行完 m-1 次 splice 后，component 数收敛为 1（得到唯一全局 cycle）；剪开任意一条边得到 AC 的 SFC path。

2.0.10.5 说明（partial correctness vs total correctness） 本文对高维合并的关键“卡点”采用严格区分：

- **partial correctness**：在任意一次 splice 被执行时，A10.2 给出其对覆盖与 AC 的保持性；因此若合并流程成功结束，
  AC。
- **total correctness**：若要证明算法对任意输入与任意合并顺序都必然成功结束，需要额外证明“每一步合并时刻
  matched-portal 必存在”。目前该性质仅以条件式写入 A10.3（并通过 A7.4 的工程诊断接口暴露失败原因），这是多

2.0.11 A11. 复杂度与可测试断言（Debug Checklist）

- Cover：O(N)
- MaxST：O(|E_{\mathrm{blk}}| \log m)，|E_{\mathrm{blk}}|\approx d\cdot m
- Merge：m-1 次 splice；portal 若缓存可摊还 O(1)，否则 face 扫描 O(2^{d-1})
- Extract/Cut：O(N)

断言：

- len(path)=N，len(set(path))=N
- \forall t,\ \|path[t+1]-path[t]\|_1=1
- 图表示中 deg(v)=2（合并中为若干 cycles，结束应为单 cycle）

2.0.12 A12. 一句话总结

在 block_size=2 的约束下，利用 BRGC 在每个 d 维超立方体 block 内构造 AC Hamiltonian cycle，并通过带符号方向的几何配
2-edge splice 沿 block-level 生成树迭代合并（结合 matched-portal 搜索与断言保证每步 splice
合法），可在任意维度 d≥2 上构造全覆盖且全局 AC 的 SFC；再借助 surrogate evaluator 将不可微解码流程纳入可训练的权

2.0.13 A13. 本仓库复现与工程接口（2D/3D）

本附录给出与本仓库代码一致的复现接口说明，便于复现实验、定位实现偏差，并将理论对象与工程对象一一对应。

2.0.13.1 A13.1 2D 最小复现脚本

仓库根目录提供：reproduce_final_2d.py。

- 输入：随机 H×W 灰度，或 --img_path 指定图片（转灰度并 resize 到 H×W，要求 H,W 为偶数）。
- 解码：Cover → Tree → Merge → Cut（对应 basic/context.py::build_circuits_graph / build_dual_graph / apply_mst）。
- 验收断言：
  - len(path)=H・W
  - len(set(path))=H・W
  - \forall t, \|path[t+1]-path[t]\|_1=1
- 输出（若启用 --save）：
  - sfc.png：图结构视角（networkx 风格）。
  - ordered_curve.png：按访问顺序绘制的单条折线曲线（更接近论文图）。

2.0.13.2 A13.2 3D 最小原型与 matched-portal 失败诊断

仓库根目录提供：reproduce_final_3d.py，用于将“2×2×2 cover + tree merge + 2-edge splice”的思想直接推广到 3D 并观察现象。

- 每个 2×2×2 block 内用 3-bit cyclic BRGC 初始化一个 8-cycle。
- 沿 block-level 生成树尝试执行 matched-portal splice。
- 在某些体数据与合并顺序下，会出现“共享面上存在 portal edge，但不存在匹配门户（matched portal）”的失败情形。

该脚本支持 --debug_fail 输出诊断信息，并在失败时保存 merge_fail.png：3D 线框展示相邻 blocks、共享面、两侧 portal edges 以及映射后未匹配的候选边，用于复现实证“matched-portal 仅能作为条件式假设或需要额外搜索/回溯自由度

2.0.13.3 A13.3 端到端流程与代码映射（img → weights → Decode → Metric）

为便于复现实验与对接学习模块，建议

- **信号（signal）**：输入图像/体数据（例如灰度图）。
- **权重（weights）**：block-level 对偶图 G_{\mathrm{blk}} 上的边权向量 w（控制生成树与合并偏好）。
- **解码器（decoder）**：确定性流程 Decode(w)，即 “Cover → Tree → Merge → Cut”。
- **指标（metric）**：对输出 SFC 计算真实目标（如 intervals/AC/TV/LZW 等）。

对应的仓库实现位置（以 2D 为主，3D 见 A13.2）：

- **Cover/Tree/Merge**：basic/context.py::build_circuits_graph、basic/context.py::build_dual_graph、basic/co
- **baseline 权重**：basic/context.py::calc_weight(..., advanced=False) 与 utils/misc.py::WeightsAssigner.calc_w A9.3）。
- **advanced 权重开关**：basic/context.py::calc_weight(..., advanced=True) 会引入更外围的邻域项（相对于 A9.3 的基础式）；训练管线中常用的是基础版。
- **真实指标（示例）**：AC/TV 等可在 basic/evaluation.py；压缩长度相关指标可在 utils/lzw.py。

另外，generate_demo_images.py 可生成多种结构化/随机灰度图作为 reproduce_final_2d.py --img_path 的输入，便于系统性对比不同纹理下的曲线行为。